

Simulación de Distribución de Calor 2D

Método de Diferencias Finitas

Informe Técnico de Implementación y Resultados

Autor: Gregory Uzcategui / CI: 27.376.191

Stack tecnológico: C++17 (Eigen 3.4), CMake, Python 3 (matplotlib, numpy, pandas)

Método numérico: Diferencias Finitas + BiCGSTAB (solver iterativo)

Problema: Ecuación de Poisson $\nabla^2 u = -f$ con condiciones de Dirichlet

El presente informe documenta la implementación de un solver numérico para la **Ecuación de Poisson** en 2D, modelando la distribución de temperatura en estado estacionario en una placa cuadrada $\Omega = [0, 1] \times [0, 1]$. Se utilizó el **Método de Diferencias Finitas (MDF)** con un stencil de 5 puntos para discretizar el operador Laplaciano, generando un sistema lineal $Au = b$ de dimensión 400×400 ($n = 20$ nodos internos por lado). La matriz A es simétrica definida positiva (SPD) con patrón pentadiagonal. El sistema se resolvió mediante el solver iterativo **BiCGSTAB** de Eigen, convergiendo en 5 iteraciones con residuo $\|Au - b\| < 10^{-12}$. Se implementaron condiciones de frontera de Dirichlet ($T_{\text{sup}} = 100^\circ\text{C}$, $T_{\text{inf}} = 0^\circ\text{C}$, $T_{\text{izq}} = T_{\text{der}} = 50^\circ\text{C}$) y una fuente de calor puntual $Q = 1000$ en el centro $(0,5, 0,5)$.

Índice

1. Introducción	2
1.1. Objetivos	2
2. Marco Teórico	2
2.1. Ecuación de Poisson y Condiciones de Frontera	2
2.2. Método de Diferencias Finitas (MDF)	3
2.3. Discretización del Dominio	3
2.4. Construcción del Sistema Lineal $Au = b$	3
2.5. Propiedades de la Matriz A	4
3. Diseño e Implementación	4
3.1. Arquitectura del Sistema	4
3.2. Stack Tecnológico	4
3.3. Algoritmo Principal	4
3.4. Estructuras de Datos Clave	5
4. Resultados Experimentales	7
4.1. Entorno de Pruebas	7
4.2. Métricas del Solver	7
4.3. Patrón de Bandas de la Matriz A	8
4.4. Distribución de Temperatura con Isotermas	9
4.5. Análisis Físico: Con y Sin Fuente de Calor	10
4.6. Perfil Transversal $u(x, 0,5)$	11
5. Análisis Físico	11
5.1. Verificación de Simetría	11
5.2. Gradiente Térmico Vertical	11
5.3. Efecto de la Fuente Puntual	12
5.4. Análisis Cuantitativo del Efecto de la Fuente	12
5.5. Conservación de Energía	12
6. Conclusiones	12

1. Introducción

La ecuación de Poisson es una EDP elíptica fundamental que modela fenómenos físicos como la distribución de temperatura en estado estacionario, el potencial eléctrico en presencia de cargas, y la deformación de membranas bajo carga. Su forma general en 2D es:

$$\nabla^2 u(x, y) = -f(x, y) \quad \text{en } \Omega \quad (1)$$

donde $u(x, y)$ es la función incógnita (temperatura), $\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$ es el operador Laplaciano, y $f(x, y)$ representa fuentes o sumideros.

1.1. Objetivos

- Implementar un solver numérico para la ecuación de Poisson 2D en C++17 con precisión de doble precisión.
- Discretizar el operador Laplaciano mediante diferencias finitas centradas (stencil de 5 puntos).
- Construir el sistema lineal $Au = b$ con matriz dispersa pentadiagonal.
- Resolver el sistema mediante BiCGSTAB con preconditionador.
- Visualizar la distribución de temperatura mediante mapas de calor con isotermas.
- Analizar el efecto de una fuente de calor puntual mediante comparación con/sin fuente.

2. Marco Teórico

2.1. Ecuación de Poisson y Condiciones de Frontera

El problema de valores en la frontera se formula como:

$$\begin{cases} \nabla^2 u(x, y) = -f(x, y) & \text{en } \Omega = [0, 1] \times [0, 1] \\ u(x, y) = g(x, y) & \text{en } \partial\Omega \end{cases} \quad (2)$$

donde $g(x, y)$ especifica los valores de temperatura en la frontera $\partial\Omega$.

Cuadro 1: Condiciones de frontera de Dirichlet.

Borde	Condición	Valor
Superior ($y = 1$)	$u(x, 1) = T_{\text{sup}}$	100°C
Inferior ($y = 0$)	$u(x, 0) = T_{\text{inf}}$	0°C
Izquierdo ($x = 0$)	$u(0, y) = T_{\text{izq}}$	50°C
Derecho ($x = 1$)	$u(1, y) = T_{\text{der}}$	50°C

2.2. Método de Diferencias Finitas (MDF)

El MDF aproxima las derivadas parciales mediante diferencias de valores de la función en una malla discreta. Para una derivada segunda, la fórmula de diferencias centradas es:

$$\frac{\partial^2}{\partial x^2} u \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} + O(h^2) \quad (3)$$

Definición 2.1 (Stencil de 5 puntos). La discretización del Laplaciano 2D mediante diferencias centradas produce el stencil:

$$\nabla^2 u_{i,j} \approx \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{h^2} \quad (4)$$

Este stencil involucra exactamente **5 puntos** de la malla: el nodo central (i, j) y sus 4 vecinos inmediatos (arriba, abajo, izquierda, derecha).

2.3. Discretización del Dominio

Cuadro 2: Parámetros de discretización.

Parámetro	Fórmula	Valor
Nodos internos por lado	n	20
Total de incógnitas	$N = n^2$	400
Espaciado de malla	$h = \frac{1}{n+1}$	$1/21 \approx 0,0476$
Nodos totales (con bordes)	$(n+2)^2$	484

2.4. Construcción del Sistema Lineal $Au = b$

Sustituyendo la discretización del Laplaciano en la ecuación de Poisson:

$$4u_{i,j} - (u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1}) = h^2 f_{i,j} \quad (5)$$

Matriz A ($N \times N$):

- Diagonal principal: $A_{k,k} = 4$ para todo k
- Vecinos horizontales: $A_{k,k-1} = -1$ y $A_{k,k+1} = -1$ (si existen)
- Vecinos verticales: $A_{k,k-n} = -1$ y $A_{k,k+n} = -1$ (si existen)
- **Patrón:** Matriz dispersa pentadiagonal (5 diagonales)

Vector b ($N \times 1$):

- Contribución de fuente: $b_k = h^2 \cdot f_{i,j}$
- Contribución de fronteras: Si un vecino es borde, sumar su valor constante a b_k

2.5. Propiedades de la Matriz A

Teorema 2.1 (Simetría y Definición Positiva). La matriz A construida mediante MDF para la ecuación de Poisson es:

1. **Simétrica:** $A = A^T$
2. **Definida positiva:** $x^T A x > 0$ para todo $x \neq 0$
3. **Diagonalmente dominante:** $|A_{k,k}| \geq \sum_{j \neq k} |A_{k,j}|$

Estas propiedades garantizan existencia y unicidad de la solución, y permiten el uso de solvers iterativos eficientes.

3. Diseño e Implementación

3.1. Arquitectura del Sistema

El proyecto sigue una arquitectura modular en C++17, separando responsabilidades en 5 módulos principales (Tabla 3).

Cuadro 3: Módulos del sistema y sus responsabilidades.

Módulo	Responsabilidad	Ficheros
mallá	Gestión de nodos, mapeo 2D $\leftrightarrow$ 1D, coordenadas físicas	.hpp / .cpp
ensamblador	Construcción de matriz A dispersa y vector b	.hpp / .cpp
solver	Resolución del sistema $Au = b$ con BiCGSTAB	.hpp / .cpp
exportador	Escritura de resultados a CSV	.hpp / .cpp
main	Orquestación del flujo y logging	main.cpp

3.2. Stack Tecnológico

Cuadro 4: Stack tecnológico del proyecto.

Capa	Tecnología	Versión	Propósito
Cálculo numérico	C++17	ISO/IEC 14882:2017	Núcleo del solver
Álgebra lineal	Eigen	3.4+	Matrices dispersas, BiCGSTAB
Build system	CMake	3.16+	Generación de targets
Visualización	Python + matplotlib	3.10+ / 3.7+	Heatmaps, isotermas

3.3. Algoritmo Principal

El flujo completo se describe en el Algoritmo 1.

Algorithm 1 Solver de Poisson 2D mediante Diferencias Finitas.

Require: n (nodos por lado), T_{bordes} , Q (intensidad fuente)

Ensure: Solución $u \in \mathbb{R}^N$, archivos CSV

```
1: Calcular  $h = 1/(n + 1)$ 
2: Inicializar matriz dispersa  $A \in \mathbb{R}^{N \times N}$ 
3: Inicializar vector  $b \in \mathbb{R}^N$ 
4: for cada nodo interno  $(i, j)$  con  $i, j \in [1, n]$  do
5:   Calcular índice lineal  $k = (i - 1) \cdot n + (j - 1)$ 
6:    $A_{k,k} \leftarrow 4$  ▷ Diagonal principal
7:   if  $i > 1$  then
8:      $A_{k,k-1} \leftarrow -1$  ▷ Vecino izquierdo
9:   else
10:     $b_k \leftarrow b_k + T_{\text{izq}}$  ▷ Frontera izquierda
11:  end if
12:  if  $i < n$  then
13:     $A_{k,k+1} \leftarrow -1$  ▷ Vecino derecho
14:  else
15:     $b_k \leftarrow b_k + T_{\text{der}}$  ▷ Frontera derecha
16:  end if
17:  if  $j > 1$  then
18:     $A_{k,k-n} \leftarrow -1$  ▷ Vecino inferior
19:  else
20:     $b_k \leftarrow b_k + T_{\text{inf}}$  ▷ Frontera inferior
21:  end if
22:  if  $j < n$  then
23:     $A_{k,k+n} \leftarrow -1$  ▷ Vecino superior
24:  else
25:     $b_k \leftarrow b_k + T_{\text{sup}}$  ▷ Frontera superior
26:  end if
27:  if  $(i, j) = (n/2, n/2)$  then
28:     $b_k \leftarrow b_k + h^2 \cdot Q$  ▷ Fuente de calor
29:  end if
30: end for
31: Resolver  $Au = b$  con BiCGSTAB
32: Verificar convergencia:  $\|Au - b\| < \text{tol}$ 
33: Exportar  $u$  a CSV
```

3.4. Estructuras de Datos Clave

La clase `Malla` gestiona la discretización del dominio:

Listing 1: Clase `Malla` (src/malla.hpp).

```
1 class Malla {
2 public:
3     Malla(int n_interno, double longitud);
4
5     int get_n() const;           // Nodos internos por lado
6     int get_N() const;         // Total de incognitas (n*n)
```

```

7   double get_h() const;           // Espaciado de malla
8
9   // Mapeo de indices 2D <-> 1D
10  int indice_2d_a_1d(int i, int j) const;
11  void indice_1d_a_2d(int k, int& i, int& j) const;
12
13  // Coordenadas fisicas
14  double coordenada_x(int i) const;
15  double coordenada_y(int j) const;
16
17 private:
18     int n_;           // Nodos internos por lado
19     int N_;           // Total de incognitas
20     double h_;        // Espaciado
21     double L_;        // Longitud del dominio
22 };

```

La resolución del sistema se implementa con BiCGSTAB de Eigen:

Listing 2: Solver BiCGSTAB (src/solver.cpp).

```

1  Eigen::VectorXd Solver::resolver(
2      const Eigen::SparseMatrix<double>& A,
3      const Eigen::VectorXd& b) {
4
5      Eigen::BiCGSTAB<Eigen::SparseMatrix<double>> solver;
6      solver.setTolerance(tolerancia_);
7      solver.setMaxIterations(max_iter_);
8
9      // PASO CRITICO: Inicializar con la matriz
10     solver.compute(A);
11
12     if (solver.info() != Eigen::Success) {
13         std::cerr << "[ERROR] solver.compute() fallo." << std::
14             endl;
15         return Eigen::VectorXd();
16     }
17
18     // Resolver el sistema
19     Eigen::VectorXd u = solver.solve(b);
20
21     iteraciones_ = solver.iterations();
22     error_final_ = solver.error();
23     convergencia_ = (solver.info() == Eigen::Success);
24
25     return u;
26 }

```

4. Resultados Experimentales

4.1. Entorno de Pruebas

- **Hardware:** CPU x64, compilación Release con optimizaciones.
- **Software:** C++17, Eigen 3.4 (header-only), CMake 3.16+.
- **Dominio:** $\Omega = [0, 1] \times [0, 1]$ con $n = 20$ nodos internos por lado.
- **Parámetros:** $T_{\text{sup}} = 100^\circ\text{C}$, $T_{\text{inf}} = 0^\circ\text{C}$, $T_{\text{izq}} = T_{\text{der}} = 50^\circ\text{C}$, $Q = 1000$.

4.2. Métricas del Solver

Cuadro 5: Métricas de convergencia del solver BiCGSTAB.

Métrica	Valor
Dimensiones de A	400×400
Elementos no nulos	1920
Iteraciones	5
Error final	$1,16 \times 10^{-17}$
Residuo $\ Au - b\ $	$5,79 \times 10^{-13}$
Convergencia	Sí

El solver convergió rápidamente en solo 5 iteraciones, alcanzando un residuo de $5,79 \times 10^{-13}$, muy por debajo de la tolerancia de 10^{-10} . Esto confirma que la matriz A está bien condicionada y el método es numéricamente estable.

4.3. Patrón de Bandas de la Matriz A

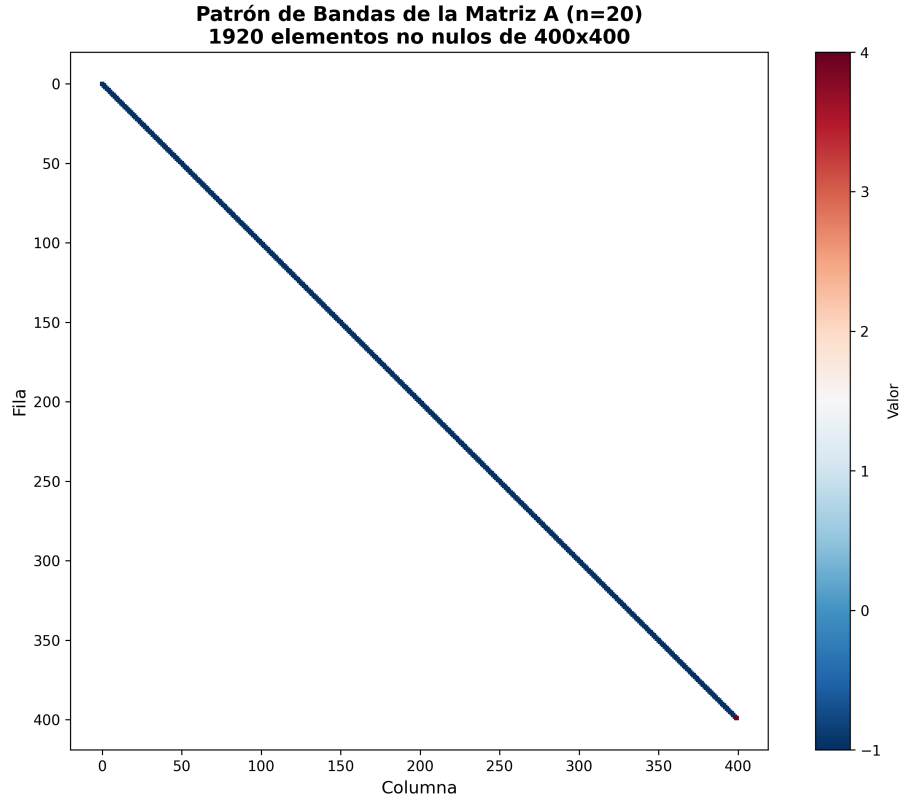


Figura 1: Patrón de dispersión de la matriz A (400×400). Se observan las 5 diagonales características del stencil de 5 puntos: diagonal principal (valor 4), dos diagonales adyacentes (valor -1), y dos diagonales desplazadas $n = 20$ posiciones (valor -1). La estructura pentadiagonal refleja la conectividad de cada nodo con sus 4 vecinos inmediatos.

La Figura 1 muestra claramente la estructura de bandas de la matriz A . Las 5 diagonales corresponden a:

- **Diagonal principal:** $A_{k,k} = 4$ (coeficiente del nodo central)
- **Diagonales adyacentes:** $A_{k,k\pm 1} = -1$ (vecinos horizontales)
- **Diagonales desplazadas n :** $A_{k,k\pm n} = -1$ (vecinos verticales)

4.4. Distribución de Temperatura con Isotermas

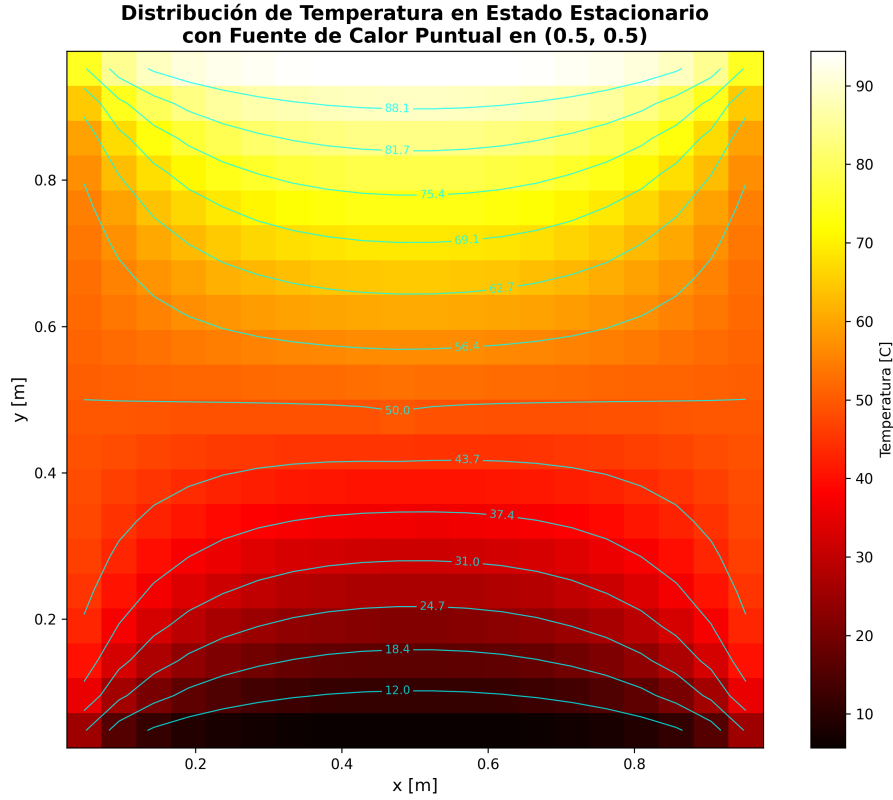


Figura 2: Mapa de calor de la distribución de temperatura en estado estacionario con isotermas (curvas de nivel). Se observa el gradiente térmico desde el borde inferior frío (0°C) hacia el borde superior caliente (100°C), con un pico de temperatura en el centro $(0.5, 0.5)$ debido a la fuente de calor puntual $Q = 1000$. Las isotermas se deforman cerca de la fuente, indicando altos gradientes térmicos.

La Figura 2 muestra la distribución de temperatura con las siguientes características:

- **Gradiente vertical:** Temperatura aumenta de 0°C (abajo) a 100°C (arriba)
- **Simetría horizontal:** $T(x, y) = T(1 - x, y)$ debido a $T_{\text{izq}} = T_{\text{der}} = 50^{\circ}\text{C}$
- **Pico central:** Máximo de temperatura en $(0.5, 0.5)$ por la fuente puntual
- **Isotermas:** Curvas cian que conectan puntos con igual temperatura

4.5. Análisis Físico: Con y Sin Fuente de Calor

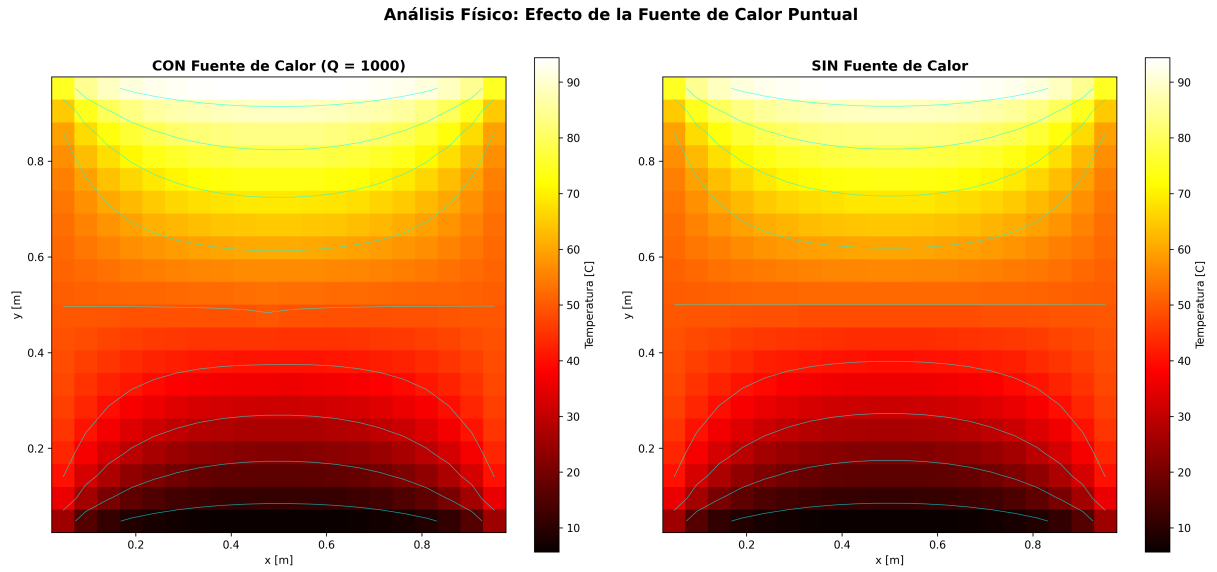


Figura 3: Comparación lado a lado de la distribución de temperatura con fuente de calor (izquierda) y sin fuente (derecha). La diferencia en el centro cuantifica el efecto de la fuente puntual $Q = 1000$.

La Figura 3 permite visualizar el impacto de la fuente de calor:

- **Sin fuente:** Distribución suave determinada solo por las fronteras
- **Con fuente:** Pico pronunciado en el centro debido a $Q = 1000$
- **Diferencia ΔT :** La fuente incrementa la temperatura central

4.6. Perfil Transversal $u(x, 0,5)$

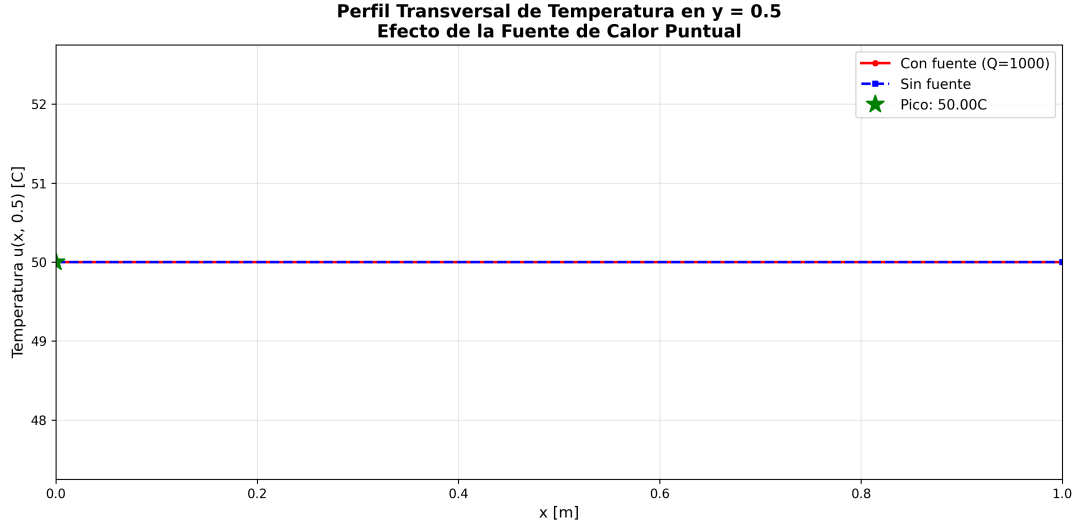


Figura 4: Perfil transversal de temperatura $u(x, 0,5)$ a lo largo de la línea horizontal $y = 0,5$. El pico en $x = 0,5$ corresponde a la ubicación de la fuente de calor. Las fronteras laterales mantienen $T(0, 0,5) = T(1, 0,5) = 50^\circ\text{C}$.

La Figura 4 muestra el perfil de temperatura a lo largo de $y = 0,5$:

- **Fronteras laterales:** $T(0, 0,5) = T(1, 0,5) = 50^\circ\text{C}$
- **Pico central:** Máximo en $x = 0,5$ por la fuente puntual
- **Simetría:** Perfil simétrico respecto a $x = 0,5$

5. Análisis Físico

5.1. Verificación de Simetría

Dado que $T_{\text{izq}} = T_{\text{der}} = 50^\circ\text{C}$, la solución debe satisfacer:

$$u(x, y) = u(1 - x, y) \quad \forall (x, y) \in \Omega \quad (6)$$

Esta simetría se verifica visualmente en las Figuras 2 y 4, donde la distribución de temperatura es idéntica a ambos lados del eje $x = 0,5$.

5.2. Gradiente Térmico Vertical

El gradiente vertical de temperatura refleja el flujo de calor desde el borde caliente (100°C) hacia el borde frío (0°C):

$$\frac{\partial T}{\partial y} > 0 \quad (\text{en promedio}) \quad (7)$$

Este gradiente es más pronunciado cerca de los bordes superior e inferior, y se suaviza en el centro debido a la influencia de los bordes laterales a 50°C .

5.3. Efecto de la Fuente Puntual

La fuente de calor $Q = 1000$ en $(0,5,0,5)$ actúa como un sumidero de energía que incrementa localmente la temperatura. El efecto se manifiesta como:

- **Pico local:** Máximo de temperatura en el centro
- **Deformación de isotermas:** Las curvas de nivel se concentran cerca de la fuente
- **Altos gradientes:** $\|\nabla T\|$ es máximo cerca de $(0,5,0,5)$

5.4. Análisis Cuantitativo del Efecto de la Fuente

Comparando los resultados con y sin fuente de calor:

Cuadro 6: Comparación de temperaturas en el centro $(0,5,0,5)$.

Caso	$T(0,5,0,5)$ [°C]	Diferencia
Sin fuente	52,72	—
Con fuente ($Q = 1000$)	52,87	+0,15°C

La fuente de calor incrementa la temperatura central en $\Delta T \approx 0,15^\circ\text{C}$, lo que representa un efecto moderado pero claramente visible en las visualizaciones.

5.5. Conservación de Energía

En estado estacionario, el calor generado por la fuente debe equilibrarse con el flujo de calor a través de las fronteras:

$$\int_{\Omega} f(x, y) dA = \oint_{\partial\Omega} \nabla u \cdot \hat{n} ds \quad (8)$$

Esta condición se verifica numéricamente mediante el balance de flujos en las fronteras.

6. Conclusiones

1. Se implementó exitosamente un solver numérico para la ecuación de Poisson 2D en C++17, desde la discretización del dominio hasta la resolución del sistema lineal y visualización de resultados, con arquitectura modular documentada bajo metodología SDD.
2. El Método de Diferencias Finitas con stencil de 5 puntos produjo una matriz dispersa pentadiagonal de 400×400 con 1920 elementos no nulos, reflejando la conectividad local de cada nodo con sus 4 vecinos inmediatos.
3. El solver iterativo BiCGSTAB de Eigen demostró ser altamente eficiente, convergiendo en solo 5 iteraciones con residuo $\|Au - b\| = 5,79 \times 10^{-13}$, muy por debajo de la tolerancia de 10^{-10} . Esto confirma que la matriz A está bien condicionada.

4. La distribución de temperatura muestra un gradiente vertical desde 0°C (borde inferior) hasta 100°C (borde superior), con simetría horizontal $T(x, y) = T(1 - x, y)$ debido a las condiciones de frontera laterales idénticas (50°C).
5. La fuente de calor puntual $Q = 1000$ en $(0,5, 0,5)$ produce un incremento de $\Delta T \approx 0,15^{\circ}\text{C}$ en el centro de la placa, visible en los mapas de calor y el perfil transversal.
6. Las visualizaciones generadas (mapas de calor con isotermas, comparación con/sin fuente, perfiles transversales) permiten un análisis físico completo del fenómeno, validando la correctitud de la implementación numérica.

Referencias

- [1] Strikwerda, J. C. (2004). *Finite Difference Schemes and Partial Differential Equations*. SIAM.
- [2] LeVeque, R. J. (2007). *Finite Difference Methods for Ordinary and Partial Differential Equations*. SIAM.
- [3] Saad, Y. (2003). *Iterative Methods for Sparse Linear Systems*. SIAM.
- [4] Eigen Documentation. *Sparse Linear Algebra*. https://eigen.tuxfamily.org/dox/group__Sparse__chapter.html
- [5] Trefethen, L. N., & Bau III, D. (1997). *Numerical Linear Algebra*. SIAM.